GO-PQ-CDC · ARCHITECTURE REVIEW

Online incremental snapshots

Engineering design RFC

Core maintainers · Adapter owners · Database engineers

> **PROPOSED — NOT IMPLEMENTED**
> Decision requested: approve bounded validation, not production release.
>
> PostgreSQL 16+ · Kafka and Elasticsearch · Restricted first scope

13 October 2026 · Revision 0.1
Source: 4a7e9b0 plus local changes
Owners and approvers: unassigned

Reading guide

1. Context and problem
2. Goals, non-goals, and support boundary
3. Correctness contract
4. Architecture and ownership
5. Source boundary and watermark protocol
6. Keyset scan and row semantics
7. Durability, checkpoints, and replay
8. Backward-compatible integration
9. Destination contracts
10. Capacity and observability
11. Alternatives and existing-draft assessment
12. Security and operations
13. Rollout, migration, and rollback
14. Verification and implementation sequence
15. Decision register and sign-off

Evidence appendices distinguish passing baseline checks from unexecuted qualification work. Source links resolve to the recorded commit and can differ from the local working tree.

RFC: online incremental snapshots

Document control	Value
Identifier / revision	RFC-ONLINE-SNAPSHOT / 0.1
Status	Proposed, not approved, not implemented
Date	2026-10-13
Purpose	Define an implementable candidate and the evidence required for acceptance
Readers	Core and adapter maintainers, PostgreSQL engineers, SRE, reviewers
Owners / approvers	Unassigned / core, database, Kafka, Elasticsearch, SRE
Companion documents	Feasibility [companion: feasibility-report.md], Validation, Evidence

Terms `MUST` and `MUST NOT` describe proposed requirements, not existing behavior. All API names and configuration fields in this RFC are proposals. Approval requires the decision register.

1. Context and problem

For normal CDC startup with `initial`, the connector creates a slot, completes a shared-view snapshot, and then opens logical replication. The slot retains WAL during the entire snapshot. The exported MVCC snapshot can also delay vacuum cleanup. Source functions and evidence are in Evidence.

The new mode must interleave short table reads with committed CDC. It must retain legacy mode behavior and public compatibility. A watermark algorithm alone does not establish safe acknowledgements or restart behavior.

2. Goals, non-goals, and support boundary

Goals: Current-state bootstrap during live CDC, no stale READ overwrite, checkpointed recovery, explicit resource limits, and unchanged legacy mode selection.

Dimension	Proposed first production profile
PostgreSQL	16+ only after each declared major/minor family passes qualification
Read location	Same primary and source identity as the replication session
Keys	Immutable, single-column `smallint`, `integer`, or `bigint` primary key
Tables	Ordinary permanent tables, no partitions, inheritance, or row-level security
Projection	One source row to one target identity, without joins or external enrichment
Publications	All required row operations and columns, with primary key available
Row images	Verified complete images, including unchanged TOAST reconstruction
Concurrency	One active source owner and one in-flight chunk window
Outputs	Kafka adapter or Elasticsearch adapter, each independently qualified
Delivery	At-least-once transport with order-aware replay handling
Protocol	Qualify protocol 1 plus `messages=true` first. Gate protocol 2 separately on commit and buffer tests

Non-goals: A global T0 snapshot, cross-table atomic target visibility, exactly-once arbitrary callbacks, distributed snapshot workers, automatic source failover, and unlimited outage tolerance.

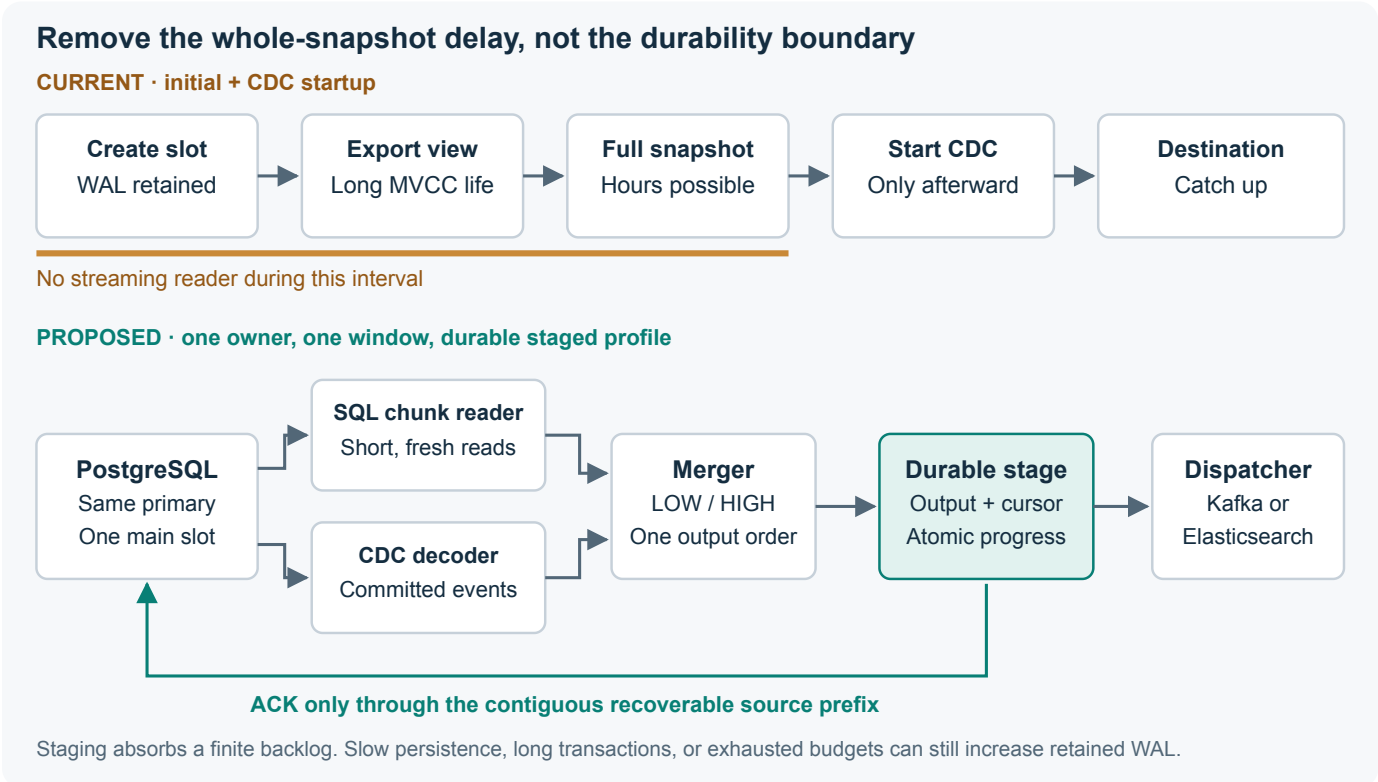
Initially reject snapshot filters, publication row filters, unsupported column projections, mutable keys, binary tuple mode, two-phase decoding, and unsupported schema changes. Reject unsupported combinations at startup. On a runtime violation, fail closed without advancing past affected work. Runtime pause/restart APIs are a later extension, except operational admission control.

3. Correctness contract

ID	Required invariant
C1 Coverage	After bootstrap and a durable catch-up fence, target keys and values equal the supported source projection
C2 Precedence	A surviving READ cannot follow a newer committed same-key change in logical output order
C3 Deletes	Snapshot or replay work cannot resurrect a row after a later delete
C4 Commit	Aborted changes or transactional markers produce no output or completed progress
C5 ACK	Source confirmation never crosses required work that is not recoverable
C6 Cursor	A completed key interval has recoverable coverage before the cursor advances
C7 Replay	Restart and ambiguous replies cause documented duplicates, not gaps or stale application
C8 Ownership	An obsolete generation cannot mutate progress or deliver accepted output
C9 Capacity	Resource use remains within declared budgets under the qualified workload envelope

C1 is evaluated at a controlled comparison fence, not against a moving source. C2 describes the logical stream. C7 requires the destination to reject or safely reconcile older transport replay. Idempotent repetition of one upsert is not sufficient protection against an older upsert.

4. Architecture and ownership



The SQL reader returns rows only to the merger. The replication reader returns commit-aware changes and control events only to the merger. Neither path invokes the public listener independently in online mode.

Component	Responsibility and ownership
Source session	Identify source, own one slot, decode transactions and controls, send bounded-time feedback
Chunk reader	Run cancellable, short SQL reads with an indexed keyset cursor
Merger	Own window state in one goroutine and determine logical output order
Progress store	Atomically preserve ordered output, source cursor, and completed chunk state
Dispatcher	Deliver staged output in order and persist destination progress
Admission controller	Limit new reads from lag, source impact, bytes, and outage budgets

Go channels MUST support cancellation. A queue's element count is not a byte bound. The merger MUST NOT perform a blocking sink request. A feedback scheduler MUST remain able to send the latest safe position during continuous traffic and bounded output backpressure.

5. Source boundary and watermark protocol

5.1 Marker transport

Use `pg_logical_emit_message(true, prefix, payload)` in separate, short transactions. Include connector ID, generation, table identity, chunk ID, unique attempt ID, and LOW/HIGH phase in a versioned payload. Enforce a payload size limit and validate its schema.

Enable `messages=true` independently of in-progress transaction streaming. Decode `M` with its optional streamed XID only when the transaction is actually streamed. Commit-gate marker actions and discard them on abort. Unrelated prefixes are not this connector's controls. Prefixes are routing conventions, not authentication.

The SQL-returned marker LSN, marker record LSN, transaction commit/end LSN, and confirmed source position are distinct. A marker return value or arrival alone MUST NOT advance source ACK.

5.2 Non-stale LOW barrier

A commit record can reach logical decoding before the transaction leaves PostgreSQL's process array. A later marker's visibility does not prove that every preceding decoded transaction is SQL-visible.

Before a chunk SELECT, the merger MUST consume its committed LOW through a visibility barrier covering every preceding relevant committed transaction. Reuse `visibilityGuard` in fail-closed mode only after validating this placement. Waiting for only the last XID is insufficient. The SELECT then obtains a fresh `READ COMMITTED` statement snapshot on the same primary.

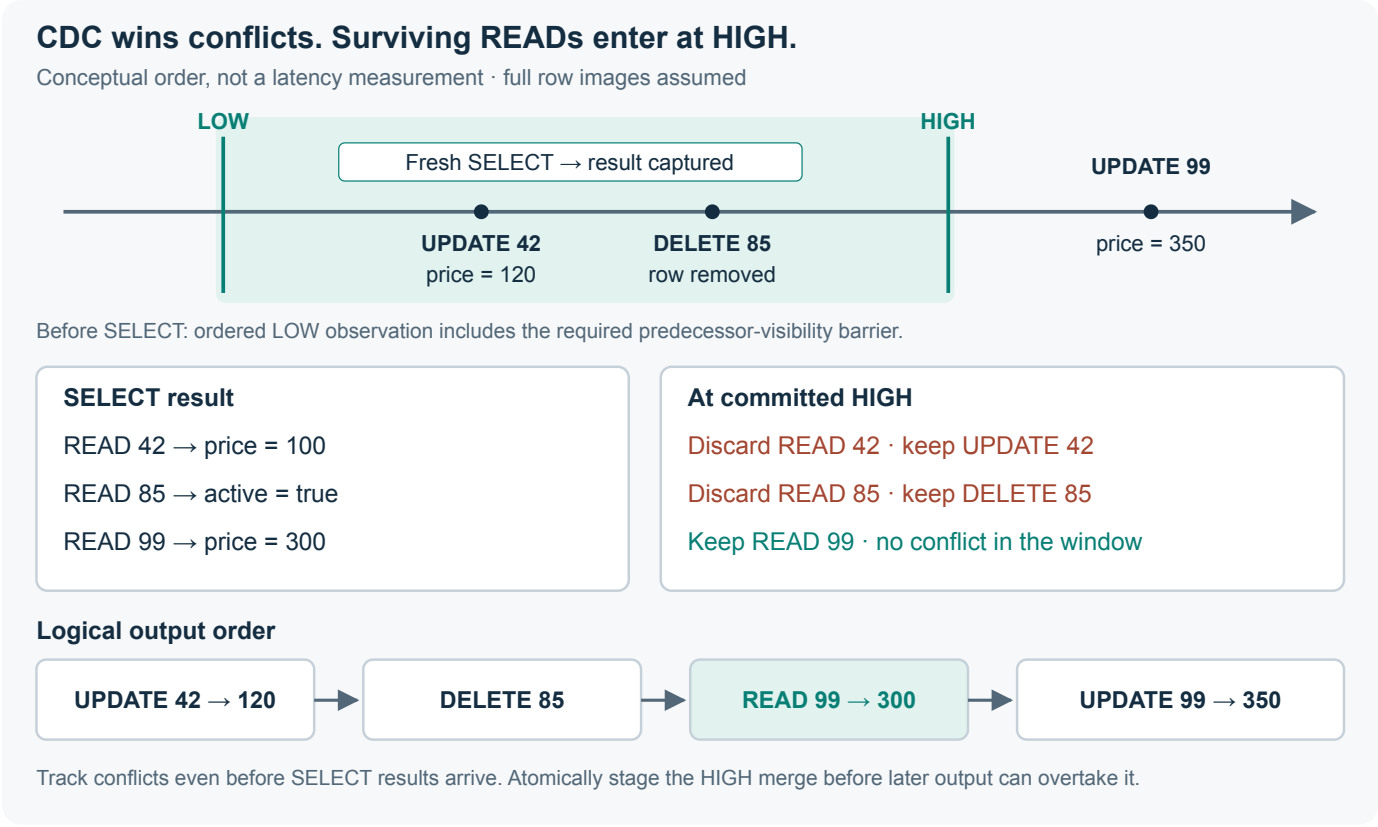
Every relevant committed transaction MUST pass visibility before its durable consumed-source cursor advances, including transactions inside an open window. This prevents ACK and restart from forgetting a transaction that remains SQL-invisible. The first profile gates those transactions before their atomic stage append.

This is a candidate proof obligation, not a claim that the current guard already implements online boundaries. Tests must force delayed visibility for an earlier commit while a later commit becomes visible. If the barrier fails this test, D2 remains blocked.

5.3 One-window algorithm

1. Validate source identity, schema fingerprint, key support, and available budgets.
2. Persist a unique chunk attempt for the current completed cursor. Register its window before emitting LOW.
3. Emit and commit LOW. Wait for ordered LOW observation and the visibility barrier.
4. Start a fresh keyset SELECT. While it runs, record relevant keys from committed CDC after LOW. Append CDC in its normal order.
5. Capture the complete selected result. Preserve the last selected key, including rows that will later lose conflicts.

6. Emit and commit HIGH. Continue CDC conflict tracking until committed HIGH reaches the merger.
7. Remove every selected row whose identity occurs in the window's touched-key set. Keep every required CDC event.
8. Conditionally complete the active attempt. Atomically append surviving READs at HIGH and preserve the completed cursor plus source progress.
9. Allow post-HIGH events to follow the READs. Advance source confirmation only through the recoverable prefix.



The HIGH boundary is a logical ordering barrier, not necessarily a wait for remote delivery. The staged profile waits for a durable append. A direct profile must instead protect pending delivery and recovery obligations.

CDC can arrive before SELECT results. Touched-key tracking therefore starts at LOW, not when rows arrive. It includes table identity, not only primary-key bytes. Transactions that start before HIGH but commit after HIGH remain later CDC.

The original DBLog algorithm pauses event processing around each small read. This RFC extends it to concurrent reads and explicitly budgets touched-key state. It does not attribute this extension or its recovery guarantees to the paper.

---

6. Keyset scan and row semantics

Capture begins before table enumeration. Capture a per-table upper key bound after that start, and persist it for this job generation. This bound makes the scan finite. Inserts behind the cursor or beyond the bound are covered by CDC.

```
-- Illustration for a supported table. Bind values and quote identifiers safely.
SELECT id, name, price
FROM public.products
WHERE id > $1 AND id <= $2
ORDER BY id
LIMIT $3;
```

The first query has no lower predicate. Do not invent a numeric sentinel that excludes the minimum key. An empty table still needs an explicit bootstrap completion fence. Checkpoint the last selected key, not the last emitted READ. Handle an all-conflict chunk without repeating it forever.

OFFSET and CTID are not online cursors. Reuse type decoders only after verifying full-image semantics. An unchanged TOAST marker is not SQL NULL. `REPLICA IDENTITY FULL` is a proposed compatibility policy, not a substitute for a full-image test. Missing required values MUST block output, not create an incomplete document.

Reject tables with row-level security, including policies that hide unchanged rows. Set reader sessions to `row_security=off` as a fail-closed safeguard, not as a policy bypass. Validate full reader visibility, not only SELECT permission.

PK mutation, TRUNCATE, and relation/schema changes invalidate the restricted profile. Do not silently drop those events. Stop before affected ACK progress and require an operator-approved schema migration or target-generation rebuild. Source-side policy must prevent unsupported DDL during qualification and production backfill.

---

7. Durability, checkpoints, and replay

7.1 Recommended staged profile

Use a transactional progress store that survives the claimed process/node failures. Persist ordered output and its consumed-source cursor in the same atomic operation. At HIGH, include surviving READs and the completed chunk cursor in that operation. The exact backend is unresolved D3.

Logical storage records:

Record	Minimum state
Source checkpoint	Source system/database identity, slot identity, generation, durable transaction-end position
Snapshot job	Mode, table/schema/key fingerprints, upper key bound, last completed selected key
Attempt	Unique attempt ID, generation, LOW/HIGH identity, lifecycle status
Ordered output	Stable output sequence, source event identity, operation, key, payload, required destinations
Delivery checkpoint	Contiguous confirmed output sequence per destination or ordered lane

CDC identity requires transaction identity and an event ordinal, not only `XLogData.WALStart`. A snapshot READ has a generation/chunk/key identity and an output sequence. Do not invent a source commit LSN for a READ. Persist schema information required to decode replayed records.

Persist each committed transaction's outputs and source progress atomically, with a bounded/spillable assembly policy. This does not require atomic visibility of that transaction at the target. Partial transaction persistence MUST NOT advance the committed source cursor.

7.2 ACK frontier

The safe frontier is a contiguous recoverable prefix, not `max(completed LSN)`. All required outputs and control obligations participate. Heartbeats cannot skip gaps. Explicit batch completion acknowledges all covered obligations, not just the final callback.

In the staged profile, visibility-gated durable CDC appends can advance source progress while a SELECT remains volatile. This is safe only because no READ from that attempt is emitted before its atomic HIGH append. On restart, discard that incomplete attempt and re-read from the previous completed key using new markers. Existing committed output remains ordered.

Every SELECT result, marker, and asynchronous completion MUST carry its unique attempt ID. Completion and abandonment are mutually exclusive conditional store transitions for that attempt. Resolve an ambiguous append before retrying. Reject late work from an abandoned attempt, even within the same owner generation.

If required stage state is absent after the source ACK advanced, stop and declare a recovery incident. Do not recreate a slot or silently choose the latest source LSN.

7.3 Crash matrix

Failure point	Required restart action
LOW or SELECT incomplete	Abandon attempt, preserve committed CDC, start a fresh attempt from the last completed cursor
HIGH known, append confirmed absent, attempt abandoned	Reject late attempt work. Retry with fresh markers and read
Atomic append committed, reply lost	Resolve by stable operation identity. Do not append READs twice after later CDC
Append committed, source ACK lost	Deduplicate source replay against the durable cursor and event identities
Sink write committed, reply lost	Retry the same output identity in order, or reject stale versions at the destination
Delivery checkpoint committed	Resume after that contiguous destination checkpoint
Progress store lost	Fail closed. Execute an approved rebuild into a clean target generation

A dispatcher MUST finish older pending work before newer same-key work can overtake it. Unknown in-flight writes from an old process require fencing or destination-enforced version order. Persist delete precedence for the supported replay horizon. A delete that discards all version evidence can permit resurrection.

7.4 Direct-delivery alternative

A direct profile avoids a stage but still requires durable progress and replay-safe destination behavior. It MUST retain source replay obligations until both output and checkpoint recovery are safe. A sink outage then retains WAL. Do not market direct and staged profiles as equivalent outage protection. D3 selects the first release profile.

8. Backward-compatible integration

Add `snapshot.mode: incremental`. Keep all current modes and defaults. Do not change the meaning of an existing `initial` configuration. Use separate versioned metadata for online jobs. Reject mode changes for an incomplete job without an explicit migration or rebuild.

Proposed additive API: an online connector/constructor with a context-aware delivery interface and explicit event/batch completion. Keep `ListenerFunc` unchanged for legacy callers. An online adapter MUST negotiate the new completion contract before startup. A legacy callback with no completion guarantee MUST NOT silently qualify.

Separate `capture ready`, `bootstrap complete`, and `delivery caught up`. Start Kafka/Elasticsearch batch loops before online reads. Existing `snapshot_only` already starts adapter batching before snapshot execution and creates no replication slot. Preserve existing readiness behavior in legacy modes. Propagate terminal errors to the online caller rather than using silent returns or process-wide fatal exits.

Integration seam	Proposed change
`config/config.go`	Mode, capability checks, limits, and invalid-combination errors
`connector.go`	Online lifecycle path without the completed-snapshot startup barrier
`pq/message/format/`	Safe logical-message decoder and tests
`pq/replication/`	Commit-aware controls, visibility placement, periodic feedback, progress interface
New `pq/snapshot/incremental/`	Keyset scanner, pure merger state machine, generation coordination
New internal delivery/store boundary	Atomic staged append, replay identity, destination progress

Integration seam	Proposed change
Adapter repositories	Batch completion, full-image mapping, ordering, lifecycle changes

9. Destination contracts

Kafka requires stable keys and partition routing, broker-confirmed writes under the approved durability settings, and ordered retry. READ and CDC use the same ordered route. Independent topics do not imply cross-topic order. Producer idempotence does not alone deduplicate application replay across restarts. Consumers need stable output identity or sequence handling for that claim.

Elasticsearch: use complete-row indexing or a proven bootstrap-safe update policy. Confirm every required bulk item. A response-handler callback is not an acknowledgement. A partial failure retains its obligation. Concurrent bulks must preserve same-key order, or writes must enforce a persisted version order with safe delete precedence.

Custom mappings, scripts, and one-to-many events need a separate adapter contract. If one input produces several required outputs, completion means all of them are safe. If a deployment configures several destinations, progress must account for all required destinations.

For rebuilds, prefer a fresh Elasticsearch index plus catch-up and alias swap. For Kafka, define a new target generation or topic policy. Re-snapshotting a damaged target in place does not remove rows that were deleted before capture resumed.

10. Capacity and observability

Limits include chunk rows, chunk bytes, touched-key bytes, transaction bytes, queued output bytes, stage disk reserve, query duration, and feedback age. Before admitting another chunk, check these limits. On a window limit, safely abandon/retry or spill according to the approved policy. Never evict conflict keys and continue the same window.

Under pressure, stop new snapshot reads first. Continue capture while the safe stage has capacity. If persistence stops, hold source ACK and alert. Backpressure cannot guarantee unlimited source protection. A blocked receive/dispatch path must not starve feedback scheduling.

Track `pg_current_wal_lsn() - restart_lsn`, confirmed lag, `wal_status`, `safe_wal_size`, source disk reserve, and relevant transaction horizons. On PG16, do not assume newer slot columns exist. Probe version-specific metrics. Long transactions can pin `restart_lsn` despite advancing confirmation.

Also track oldest window, read latency, collision ratio, stage bytes/age, oldest unfinished output, sink errors, feedback age, and table progress. Use low-cardinality metric labels. Keep keys and payloads out of metric labels and routine logs.

Capacity planning uses measured stage bytes per second and source WAL generation, not source row counts alone. Approximate stage reserve is backlog growth rate multiplied by tolerated outage duration, plus a measured recovery and disk safety margin. No benchmark numbers or SLO targets are claimed by this RFC.

11. Alternatives and existing-draft assessment

Candidate	Decision rationale
Literal DBLog short dispatch pause	Simpler conflict memory. SELECT latency delays processing. Retain as a bounded prototype comparator
Concurrent logical-marker windows	Recommended candidate. One slot and explicit controls. Needs visibility, touched-key, and recovery proofs
Sample `pg_current_wal_lsn()` around SELECT	Reject without visibility and commit-boundary proof. Numeric positions do not establish MVCC visibility

Candidate	Decision rationale
Temporary slot per chunk	Keep for validation. Server snapshot contract applies to that slot's stream, not automatically to the main stream
Shared snapshot plus durable spool	Valid separate T0-style mode. Does not remove the long MVCC horizon. Backlog requires capacity
External CDC platform	Evaluate operational cost and migration. Do not infer guarantees from a product name

The existing `../concurrent-snapshot-design.md` stays unchanged. Its temporary-slot direction requires a specified mapping from exported snapshot to main-stream transactions, a completion fence, and output ordering. Temporary slot creation also needs measured latency, cleanup, and retention behavior. It can wait on source transactions.

This RFC does not accept claims of guaranteed bounded WAL, total memory equal to one chunk, or automatic safe fallback. It does not copy version-specific monitoring claims without verification. These differences are technical review findings, not evidence that either draft is implemented.

12. Security and operations

Use least-privilege SQL and replication roles, explicit marker-emission grants, TLS, and safe identifier quoting. Validate marker IDs and payload sizes. Do not include credentials or sensitive row data in marker payloads. Validate schema and publication permissions before capture.

The stage contains source data. Define encryption, access control, retention, backup, and deletion responsibilities. Keep stage traffic out of the captured publication if a PostgreSQL backend is selected. Prefer a store independent of the production source's failure domain.

Runbooks must cover sink outage, stage capacity, source reconnect, slot loss, schema violation, and progress-store loss. PostgreSQL HA and automatic slot failover remain outside v1. A source identity/timeline change requires a verified continuity decision, not an automatic LSN comparison.

13. Rollout, migration, and rollback

Ship disabled by default. Qualify one table and one target generation first. Compare complete target state at explicit fences. A shadow test uses a separate test slot and isolated destination. Do not attach two readers to one slot.

Pause new reads without dropping CDC obligations. Keep stage and source checkpoints during rollback. Drain with a compatible binary, or rebuild a clean target generation from an approved boundary. Do not switch an incomplete online job into legacy mode automatically. Slot invalidation requires incident handling, not silent recreation.

Expand protocols, key types, schemas, and concurrency only after separate qualification. Validation is the release gate, not an optional appendix.

14. Verification and implementation sequence

Implement P0 baseline tests, P1 pure-model validation, P2 restricted source protocol, P3 stage and adapters, P4 operational qualification, and P5 canary. Each phase has an evidence gate in the feasibility report. Do not schedule a production date before P1 resolves D2 and bounds D3.

Document verify: `test -s docs/online-snapshot/validation-plan.md && test -s docs/online-snapshot/evidence.md`. This verifies document presence only. Runtime correctness requires C1–C9 tests and recorded results.

15. Decision register and sign-off

ID	Decision	Recommendation / status	Approver role
D1	Semantics and first table/key profile	Current-state, integer immutable PK, restricted projection / proposed	Core + product
D2	Boundary protocol	Logical markers plus all-prior visibility barrier / proof pending	Database + core
D3	Durability profile and backend	Transactional durable stage / backend and failure domain open	Core + SRE
D4	Destination replay/fencing strategy	Stable sequences and ordered recovery / adapter design open	Kafka + Elasticsearch
D5	Capacity and recovery budgets	Measure production-like workload / values open	Database + SRE
D6	Public API and metadata migration	Additive online API and separate versioned state / review pending	Core + adapters

Acceptance: named approvers, resolved D1–D6, linked validation artifacts, and an approved rollback drill. No sign-off exists at revision 0.1.

SUPPORTING APPENDIX

Online snapshots: validation and qualification plan

Status: proposed acceptance plan, not completed test evidence. **Date:** 2026-10-13.

Purpose: turn RFC invariants C1–C9 into falsifiable release gates. Core, database, adapter, and SRE owners must sign their assigned evidence. Test names in this plan are proposed unless Evidence lists them as existing.

1. Test oracle

Use a persistent reference target keyed by source table and primary key. Record source operations, transaction identities, commit order, and each logical output sequence. Distinguish logical emissions from transport retries.

After bootstrap, stop writes and commit a final marker. Drain required delivery through that marker. Compare complete target keys and values against the source. Include deleted keys in the comparison. For live comparisons, first define a coordinated source view and fence.

Require zero missing keys, zero unexpected keys, zero value differences, and zero stale application. Counts alone can hide one missing row and one duplicate. Transport duplicates are permitted only when the destination handles them without stale effects.

A CDC row can supply snapshot coverage. Do not require one READ per source row. For ordinary CDC, preserve all required committed events. Do not suppress real CDC merely because a snapshot row conflicts.

2. Deterministic model and fault scheduler

Implement a pure merger model before a database connector. Begin with three keys, two chunks, two source transactions, a persistent fake stage, and a fake destination. Enumerate legal schedules. Then add seeded random schedules with minimized counterexamples.

The scheduler controls these boundaries:

Boundary	Controlled action
Source commit	Publish a committed event or abort a streamed transaction
Visibility	Delay SQL visibility independently of decoded commit order
LOW	Commit and observe the exact marker after earlier relevant visibility checks
SELECT	Acquire the statement snapshot and return results as separate actions
HIGH	Commit and observe the exact marker after result capture
Stage	Commit an append, fail it, or lose its success reply
Destination	Complete selected items, fail items, delay writes, or lose responses
Progress	Persist cursor/delivery state or fail after the write
Restart	Destroy all volatile state and rebuild from durable records
Ownership	Replace the generation while the former owner still has pending work

Never model a crash as graceful `Close`. Never retain the touched-key set or SELECT result across a simulated process death unless the specified store persists it.

3. Traceability matrix

Test ID	Required scenario	Invariants	Pass evidence
T01	Update after LOW, before SELECT result arrives	C1, C2	CDC remains, stale READ absent
T02	Delete during the read, then delete/reinsert	C1–C3	No resurrection, later insertion wins
T03	Every selected row conflicts	C1, C6	Cursor advances from selected keys, no endless repeat
T04	Empty table, empty final page, minimum and sparse integer keys	C1, C6	Complete coverage and finite completion
T05	Transaction begins before HIGH, commits afterward	C2, C4	Its committed change follows surviving READs
T06	Streamed marker/data abort and subtransaction rollback	C4	No aborted output or progress
T07	Same key in another table, duplicate marker, obsolete generation or abandoned attempt	C2, C8	Reject late SELECT/HIGH results, including same-generation retries
T08	Later ACK and heartbeat while earlier output is unfinished	C5	Safe frontier cannot cross the gap
T09	Crash at every transition in section 2	C1–C7	Recovery preserves exact state and safe cursor
T10	Stage commit succeeds but reply or source ACK is lost	C5–C7	Resolve ambiguous attempt completion before retry, no reordered stage output
T11	Destination write succeeds but response is lost	C2, C3, C7	Ordered retry or version rejection prevents stale effects
T12	Old owner sends late metadata and destination writes	C7, C8	New owner cannot be corrupted by stale work
T13	Earlier commit stays SQL-invisible while a later one is visible, including an ACK/crash/restart schedule	C1, C2, C5	Visibility gates survive restart through the durable-cursor precondition
T14	Large unchanged TOAST, NULL transitions, UPDATE before baseline	C1	Complete correct image or explicit unsupported failure
T15	PK mutation, TRUNCATE, DDL, filters, RLS with an unchanged hidden row	C1–C6	Explicit failure before unsafe ACK, no silent partial support
T16	Maximum chunk, transaction, touched-key, and stage budgets	C5, C9	Bounded resources, no unsafe eviction or drop
T17	Continuous input without receive-idle gaps	C5, C9	Feedback progresses on an explicit bounded cadence
T18	Walsender loss, blackholed connection, stalled feedback write	C5, C7, C9	Bounded shutdown/recovery and no skipped obligations
T19	Lost slot or missing stage state	C5, C7	Fail closed with an actionable incident, no silent slot recreation
T20	Retained legacy modes and public callbacks	Compatibility	Existing configurations and semantics remain supported

Run the model under `go test -race`. Add fuzz targets for marker framing, lengths, streamed-XID handling, tuple presence, and key encoding. Malformed input must not panic or allocate unbounded memory. Distinguish intentionally ignored messages from failed decoding of required events.

4. PostgreSQL integration matrix

Qualify PostgreSQL 16 and each additional major advertised by the release. Pin exact test images or digests in release evidence. Do not interpret PostgreSQL 16+ as automatic certification of future releases.

Dimension	Required profiles
Wire protocol	Protocol 1 plus logical messages. Protocol 2 only after its stream/abort/buffer gates pass
Durability	Functional fast profile and separate `fsync=on` crash profile with durable volumes
Authentication	Restricted replication/SELECT/marker role, not only superuser
Transactions	Small commits, large commits, rollback, savepoint rollback, long-lived source transactions
Read timing	Explicit SELECT barriers with writes inside the actual window
Visibility	Synchronous-replication fixture or equivalent deterministic server instrumentation for T13
Lifecycle	Hard connector kill, PostgreSQL crash/restart, reconnect, stage restart
Retention	Small controlled WAL allowance, forced checkpoints, retained-WAL observation and slot loss
Identity	Startup identity check and fail-closed behavior on identity/timeline mismatch

Verify that the intended wire path actually occurred. A protocol-2 test that never observes streamed messages does not cover stream handling. Test helper processes must dispatch their special mode before normal test setup.

Automatic promotion continuity is outside the first profile. If later supported, test actual promotion with correctly synchronized slot state. A test that only reads a failover flag is insufficient.

5. Adapter contract suite

Apply the same fault scenarios to both adapters. Use real Kafka and Elasticsearch for final qualification, not only mocks.

Contract	Kafka evidence	Elasticsearch evidence
Durable success	Approved broker ACK/replication policy and confirmed producer write	Every required bulk item succeeds under approved durability settings
Stable identity	Same source identity uses a stable partition route	Same source identity uses a stable document ID and routing
Ordering	READ and CDC retain order across batches and retries	Concurrent bulks cannot apply older same-key work after newer work
Partial failure	Failed records remain unresolved	Failed items remain unresolved even with an HTTP success
Replay	Restarted producer/consumer handles stable output IDs and sequences	Stale retries and deletes respect the version/tombstone policy
Expansion	All outputs from one source event participate in completion	All actions from one source event participate in completion
Lifecycle	Producer batch loop starts before online capture can block	Bulk loop starts before online capture can block

A custom error handler that logs and returns MUST NOT turn failure into success. Oversized required records and mapping failures must remain unresolved or trigger an explicit operator decision. A dead-letter policy changes the zero-loss-to-target contract and requires separate approval.

6. Capacity experiment

Measure the legacy mode and the candidate under the same database size, row widths, source resources, write mix, and destination limits. Record source latency before the experiment. Use an isolated database, not a production source.

Include sustained inserts, updates, deletes, a hot-key workload, wide TOAST values, long transactions, and a bounded destination outage. Run for at least the target snapshot duration plus the recovery interval. Record peak usage, not only memory after garbage collection.

Budget	Value required before P4	Measurement
Source workload	Writes/s, WAL bytes/s, transaction-size distribution	Source workload recorder and WAL deltas
Snapshot workload	Rows, bytes, chunk row/byte caps, maximum query duration	Query timing and scanner counters
CDC service level	Allowed percentile latency and feedback age	Source commit-to-durable-output timing
Memory and disk	Heap/RSS peak, stage capacity and disk reserve	Process and store telemetry
Source impact	Allowed query latency, CPU and I/O increase	Comparison against baseline
Outage tolerance	Maximum sink and stage outage durations	Controlled fault injection
Recovery	Maximum catch-up time after the outage	Durable destination checkpoint lag
Retention	Allowed source WAL and transaction horizon	`restart_lsn`, `confirmed_flush_lsn`, slot and backend views

All values are **TBD by database/SRE owners**. No production benchmark is available. Stage ingress must keep up with measured CDC plus READ output during normal operation. Catch-up capacity must exceed new arrival rate after an outage.

Use the source's total retained WAL, not only captured table activity. `active=true` and advancing `confirmed_flush_lsn` are not sufficient: long transactions can hold `restart_lsn` back.

7. Release evidence and gates

Gate	Required artifact	Sign-off
G0 Baseline	Exact legacy assertions, ACK-gap reproduction, source revisions	Core
G1 Algorithm	Model schedules, seeds, minimized counterexamples, C1–C9 results	Core + database
G2 Source protocol	Versioned wire traces and real LOW/read/HIGH/abort tests	Database
G3 Recovery	Hard-crash traces, durable state before/after, replay results	Core + SRE
G4 Destinations	Kafka and Elasticsearch partial-failure/replay reports	Both adapter owners
G5 Capacity/security	Signed budgets, measured peaks, restricted-role results	Database + SRE
G6 Rollout	Canary comparison, alert tests, rollback and rebuild drill	Service owner + SRE

Archive the exact code and configuration revisions, database image digests, test commands, logs, workload seeds, and expected/actual state differences. Preserve failed cases as regression fixtures. Require independent review of the recovery state machine before release.

8. Verification commands and next action

Existing baseline commands, not online qualification:

```
go test -race -count=1 ./...
go vet ./...
(cd integration_test && POSTGRES_VERSION=16-alpine go test -race -count=1 -p=1 ./...)
```

The integration module is separate. Root `go test ./...` does not run it. The full integration command in this section is a proposed gate. The full suite was not executed during this assessment.

Next action: Assign owners for T08, T09, and T13. Implement those falsifiers and the pure model before wiring the online mode into `connector.Start`.

Document verify: `grep -c '^| T[0-9][0-9]' docs/online-snapshot/validation-plan.md` must report `20`. This verifies traceability entries, not executed tests.

SUPPORTING APPENDIX

Online snapshots: evidence and provenance

Date: 2026-10-13. **Purpose:** separate source observations, executed checks, published mechanisms, and proposed guarantees.

1. Repository baseline

Repository	Inspected revision	Qualification
`go-pq-cdc`	`4a7e9b094a19935591b691cf8de38546a98b69d0`	Branch `fix/slot-info-reconnect-on-closed-conn`, including local changes and new tests
`go-pq-cdc-kafka`	`28f6072c4924bb7a8c2244128423b0bd64ab340b`	Local adapter source inspection, not runtime qualification
`go-pq-cdc-elasticsearch`	`5d8774443ff36c0b62c12f1bc9dbde2469e87cfe`	Local adapter source inspection, not runtime qualification

The checkout was not clean. The root commit alone does not reproduce these results. Baseline output [companion: baseline-checks.txt] records the tracked working-diff digest and toolchain. Untracked tests also participated. No Go implementation file was changed for this document package.

The existing concurrent snapshot draft [companion: ../concurrent-snapshot-design.md] and presentation proposal [companion: ../presentations/online-snapshot/README.md] remain unchanged. They are design input, not deployed-feature evidence. This package independently compares their proposals.

2. Source observations

Claim	Source location	Classification
Snapshot completes before replication opens	`connector.Start` and `prepareSnapshotAndSlot`	Implemented control flow
Coordinator holds an exported view across chunks	`exportSnapshotTransaction`, `executeInTransaction`	Implemented behavior
Current WAL sample is independent of the exported snapshot	`setupJob`, `getCurrentLSN`	Source-derived overlap risk, no boundary reproduction in this package
Snapshot listener ACK does nothing	`snapshotHandler`	Implemented behavior
Snapshot row-handler errors are ignored	`processChunk`	Implemented behavior
CDC confirmation keeps a maximum position	`processLoop`, `UpdateConfirmedXLogPos`	Implemented behavior, insufficient for independent asynchronous completions
Logical messages are not decoded	`message.New`	Implemented gap
Message and streaming options are coupled	`Replication.Start`	Implemented behavior
Streamed transactions accumulate until commit	`streamTxBuffer`	Implemented behavior, no configured byte limit
Visibility guard checks source transaction visibility	`visibilityGuard`	Existing component, not an online-window proof

Claim	Source location	Classification
Incomplete legacy jobs can restart with a fresh view	`initializeCoordinator`	Implemented behavior, not target reconciliation
Unchanged TOAST can require reconstruction	`Update.decode`, `DecodeWithColumn`	Existing tuple semantics

Line anchors describe this inspection and can move. Function names are the durable lookup keys.

Adapter observations

- In normal CDC startup, Kafka `connector.Start` waits for core readiness before `StartBatch`. `Batch.flushMessages` uses the final listener context as a cumulative ACK convention.
- In normal CDC startup, Elasticsearch `connector.Start` waits for core readiness before `StartBulk`. `Bulk.flushMessages` can clear and acknowledge a failed batch when a custom response handler is installed.
- The separate `snapshot_only` path creates no replication slot. Both adapters start their batch loops before snapshot-only core startup.
- These observations motivate an explicit online delivery contract. They are not a complete audit of either adapter.

Adapter locations are external to this repository: `go-pq-cdc-kafka/connector.go`, `kafka/producer/producer_batch.go`, `go-pq-cdc-elasticsearch/connector.go`, and `elasticsearch/bulk/bulk.go` at the revisions in Repository baseline.

3. Executed baseline checks

The checks were repeated on 2026-10-13 during document preparation. Captured output [companion: baseline-checks.txt] records success. Toolchain: Go 1.26.2, macOS arm64. The integration harness used Podman and its PostgreSQL `16-alpine` image profile.

```
go test -race -count=1 ./...
go vet ./...
```

The separate integration module ran this selection:

```
cd integration_test
POSTGRES_VERSION=16-alpine go test -race -count=1 -p=1 -timeout=120s \
  -run
'^(TestSnapshotConcurrentInsertNoDuplicates|TestNoWalsenderDuringSnapshot|TestSnapshotBasicSingl
eChunk|TestConcurrentTxOrdering|TestStreamingSubTransactionRollback|TestSIGTERMFlushesPendingCon
firmedLSN|TestSlotInfoReconnectsAfterBackendTermination)$' ./...
```

Result	Scope
PASS	Root race-enabled tests
PASS	Root `go vet`
PASS	Seven selected PostgreSQL 16 integration tests
NOT RUN	Full integration suite and full supported-version matrix
NOT RUN	Kafka and Elasticsearch runtime suites
NOT RUN	Online merger, recovery, fault injection, sustained-load, and HA qualification

Limits of the passing tests

`TestSnapshotConcurrentInsertNoDuplicates` waits for readiness before it starts concurrent writes. Readiness currently follows the snapshot. The test accepts 45 of 51 CDC keys and at least 95 of 101 combined keys. It does not prove zero loss during a snapshot race.

`TestNoWalsenderDuringSnapshot` starts its sampling interval after readiness. It does not require an observation during actual snapshot execution.

`TestMain` uses `fsync=off`. Its CDC role is a superuser. Separate durable and restricted-role profiles are required.

The untracked disconnect test defines a helper without dispatch from `TestMain`. It was not part of the selected run. Repair its helper wiring before relying on it as crash evidence. A forced process exit alone does not prove continuity after restart.

4. Published mechanisms and their limits

Reference	Supported claim	Not established by the reference
DBLog paper, section 3.2 / Algorithm 1	LOW/SELECT/HIGH reconciliation, CDC-wins conflict removal, short per-chunk processing pause	This library's recovery protocol, bounded total memory, or WAL benchmark
PostgreSQL 16 logical replication parameters	`messages=true` and separate transaction-streaming options	Correct application dispatch or acknowledgement behavior
PostgreSQL 16 message format	`M` payload layout and optional streamed XID, unchanged TOAST representation	A complete bootstrap row from every UPDATE
PostgreSQL 16 administrative functions	Transactional logical-message emission	Permission to ACK on message emission
PostgreSQL 16 logical decoding concepts	A slot-exported snapshot corresponds to that slot's subsequent change stream	Safe substitution into a different slot's stream
PostgreSQL 16 replication commands	Slot creation results, snapshot lifetime, logical start-position rules	A reusable application crash/checkpoint protocol
PostgreSQL 16 slot view	`restart_lsn`, `confirmed_flush_lsn`, `wal_status`, and `safe_wal_size`	Guaranteed bounded WAL from slot activity alone

The paper describes watermark-table updates. Logical-message transport is a PostgreSQL-specific adaptation. The concurrent read extension, visibility barrier, durable-stage protocol, and release gates are proposals in this RFC.

Do not use older comparative tables in the paper to describe current competing products. This package does not certify the existing draft's external incident summaries or Flink implementation claims.

5. Independent review and document status

The preceding analysis used separate read-only code and reliability reviewers. This package also received independent protocol and evidence reviews. Parent revisions incorporated restart-complete visibility, attempt-level fencing, explicit RLS exclusion, and mode-qualified startup descriptions.

The reviewers did not execute tests or validate PDF rendering. Their review is not an implementation proof or formal approval. The parent checked the final documents and exports separately.

Current conclusion: feasible to validate, not approved for production. No performance improvement, recovery-time target, or zero-loss production result is claimed.

Document verify: `test -s docs/online-snapshot/baseline-checks.txt && grep -q 'All selected baseline checks: PASS' docs/online-snapshot/baseline-checks.txt`.